LAB04 : Création d'images Docker : docker build et docker commit

Préparation du code source

1- Créez le répertoire de travail :

```
chaima@chaima-VirtualBox:~$ mkdir -p ~/lab04 && cd ~/lab04
chaima@chaima-VirtualBox:~/lab04$
```

2- Créez le fichier hello.c :

```
chaima@chaima-VirtualBox:~/lab04$ cat > hello.c <<'EOF'
> #include <stdio.h>
>
> int main(void)
> {
>     printf("Hello From Docker LAB ! \n");
>     return 0;
> }
> EOF
```

3- Vérifiez le contenu avec cat hello.c :

```
chaima@chaima-VirtualBox:~/lab04$ cat hello.c
#include <stdio.h>

int main(void)
{
    printf("Hello From Docker LAB ! \n");
    return 0;
}
```

Partie 1 : image depuis Ubuntu avec docker build (version1)

1- Créez le fichier Dockerfile.ubuntu :

```
chaima@chaima-VirtualBox:~/lab04$ cat > Dockerfile.ubuntu <<'EOF'
> FROM ubuntu:24.04
>
> #Mise à jour + installation + nettoyage dans un seul RUN
> RUN apt-get update \
> && apt-get install -y --no-install-recommends gcc libc6-dev \
> && rm -rf /var/lib/apt/lists/*
>
> COPY hello.c /src/hello.c
> RUN gcc -o /hello /src/hello.c
>
> CMD ["/hello"]
> EOF
```

2- Construisez l'image :

```
chaima@chaima-VirtualBox:~/lab04$ docker build -f Dockerfile.ubuntu -t hello:version1 .
Sending build context to Docker daemon  3.072kB
Step 1/5 : FROM ubuntu:24.04
24.04: Pulling from library/ubuntu
7a48b8f0e5c7: Pull complete
Digest: sha256:fe81560990fae86dc4743202b2cb4d3df98d0bcf949173cecef15ce1127ef852
Status: Downloaded newer image for ubuntu:24.04
 ---> d9b8c34d61a9
Successfully built eca112c837bb
Successfully tagged hello:version1
```

3- Vérifiez et exécutez :

```
chaima@chaima-VirtualBox:~/lab04$ docker images hello
REPOSITORY   TAG        IMAGE ID       CREATED          SIZE
hello        version1   eca112c837bb   54 seconds ago   266MB
chaima@chaima-VirtualBox:~/lab04$ docker run --rm hello:version1
Hello From Docker LAB !
```

4- Affichez les couches de l'image :

```
chaima@chaima-VirtualBox:~/lab04$ docker history hello:version1
IMAGE          CREATED         CREATED BY                                      SIZE      COMMENT
eca112c837bb   2 minutes ago   /bin/sh -c #(nop)  CMD ["/hello"]               0B
acc103a8e1f9   2 minutes ago   /bin/sh -c gcc -o /hello /src/hello.c           16kB
c70a60aa5199   2 minutes ago   /bin/sh -c #(nop) COPY file:616b7bfff4e63549…   95B
cbe421506eb0   2 minutes ago   /bin/sh -c apt-get update && apt-get install…   188MB
d9b8c34d61a9   12 days ago     /bin/sh -c #(nop)  CMD ["/bin/bash"]            0B
<missing>      12 days ago     /bin/sh -c #(nop) ADD file:eed88e94d6aa7c9da…   78.2MB
<missing>      12 days ago     /bin/sh -c #(nop)  LABEL org.opencontainers.…   0B
<missing>      12 days ago     /bin/sh -c #(nop)  ARG LAUNCHPAD_BUILD_ARCH     0B
<missing>      12 days ago     /bin/sh -c #(nop)  ARG RELEASE                  0B
```

5- Relancez exactement la même commande docker build :

```
chaima@chaima-VirtualBox:~/lab04$ docker build -f Dockerfile.ubuntu -t hello:version1 .
Sending build context to Docker daemon  3.072kB
Step 1/5 : FROM ubuntu:24.04
 ---> d9b8c34d61a9
Step 2/5 : RUN apt-get update && apt-get install -y --no-install-recommends gcc libc6-dev && rm
-rf /var/lib/apt/lists/*
 ---> Using cache
 ---> cbe421506eb0
Step 3/5 : COPY hello.c /src/hello.c
 ---> Using cache
 ---> c70a60aa5199
Step 4/5 : RUN gcc -o /hello /src/hello.c
 ---> Using cache
 ---> acc103a8e1f9
Step 5/5 : CMD ["/hello"]
 ---> Using cache
 ---> eca112c837bb
Successfully built eca112c837bb
Successfully tagged hello:version1
```

Remarque : le build est quasi instantané et chaque étape affiche CACHED. Docker réutilise une couche tant que l'instruction et les fichiers copiés n'ont pas changé. Si tu modifies hello.c, seules les étapes à partir de COPY sont refaites.

Questions

1. Pourquoi apt-get update et apt-get install sont-ils dans le même RUN ? Que risquerait-on avec deux RUN séparés si l'on ajoutait un paquet plus tard ?

→ Avec deux RUN séparés, la couche apt-get update resterait en cache. Si on ajoute plus tard un paquet à l'install, Docker réutiliserait un index de paquets périmé, ce qui peut produire des versions obsolètes ou des erreurs 404. Les regrouper garantit un index frais à chaque modification. En plus, le rm -rf /var/lib/apt/lists/* dans le même RUN évite de garder ces listes dans une couche.

2. Pourquoi faut-il installer libc6-dev en plus de gcc quand on utilise --no-install-recommends ? (Indice : où se trouve stdio.h ?)

→ stdio.h se trouve dans libc6-dev. Normalement gcc le tire en *recommandation*. Avec --no-install-recommends, il n'est pas installé, donc il faut le demander explicitement, sinon on obtient fatal error: stdio.h: No such file or directory.

3. Pourquoi écrire ubuntu:24.04 plutôt que ubuntu ?

→ ubuntu équivaut à ubuntu:latest, qui change avec le temps (24.04 aujourd'hui, 26.04 demain). Fixer 24.04 rend le build reproductible et évite les surprises.

Partie 2 : image minimale scratch par build multi-étapes (version2)

1- Créez le fichier Dockerfile.scratch :

```
chaima@chaima-VirtualBox:~/lab04$ cat > Dockerfile.scratch <<'EOF'
> FROM alpine:3.20 AS build
> RUN apk add --no-cache gcc musl-dev
> COPY hello.c /src/hello.c
> RUN gcc -static -o /hello /src/hello.c
>
> FROM scratch
> COPY --from=build /hello /hello
> CMD ["/hello"]
> EOF
```

2- Construisez et exécutez :

```
chaima@chaima-VirtualBox:~/lab04$ docker build -f Dockerfile.scratch -t hello:version2 .
Sending build context to Docker daemon  4.096kB
Step 1/7 : FROM alpine:3.20 AS build
3.20: Pulling from library/alpine
25f1d6b1951a: Pull complete
Digest: sha256:d9e853e87e55526f6b2917df91a2115c36dd7c696a35be12163d44e6e2a4b6bc
Status: Downloaded newer image for alpine:3.20
 ---> bf8527eb54c3
Step 2/7 : RUN apk add --no-cache gcc musl-dev
 ---> Running in 52653c143851
fetch https://dl-cdn.alpinelinux.org/alpine/v3.20/main/x86_64/APKINDEX.tar.gz
fetch https://dl-cdn.alpinelinux.org/alpine/v3.20/community/x86_64/APKINDEX.tar.gz
(1/14) Upgrading musl (1.2.5-r3 -> 1.2.5-r4)
(2/14) Installing libgcc (13.2.1_git20240309-r1)
(3/14) Installing jansson (2.14-r4)
(4/14) Installing libstdc++ (13.2.1_git20240309-r1)
(5/14) Installing zstd-libs (1.5.6-r0)
(6/14) Installing binutils (2.42-r1)
(7/14) Installing libgomp (13.2.1_git20240309-r1)
(8/14) Installing libatomic (13.2.1_git20240309-r1)
(9/14) Installing gmp (6.3.0-r1)
(10/14) Installing isl26 (0.26-r1)
(11/14) Installing mpfr4 (4.2.1-r0)
(12/14) Installing mpc1 (1.3.1-r1)
(13/14) Installing gcc (13.2.1_git20240309-r1)
(14/14) Installing musl-dev (1.2.5-r4)
Executing busybox-1.36.1-r31.trigger
OK: 157 MiB in 27 packages
Removing intermediate container 52653c143851
 ---> e1283ad01d67
Step 3/7 : COPY hello.c /src/hello.c
 ---> 81fdbbb94757
Step 4/7 : RUN gcc -static -o /hello /src/hello.c
 ---> Running in 06d3da6027ee
Removing intermediate container 06d3da6027ee
 ---> d1799e3c7b8c
Step 5/7 : FROM scratch
 --->
Step 6/7 : COPY --from=build /hello /hello
 ---> 2431f3bc86b8
Step 7/7 : CMD ["/hello"]
 ---> Running in b58150d15c06
Removing intermediate container b58150d15c06
 ---> 37e759ef1d58
Successfully built 37e759ef1d58
Successfully tagged hello:version2
chaima@chaima-VirtualBox:~/lab04$ docker run --rm hello:version2
Hello From Docker LAB !
chaima@chaima-VirtualBox:~/lab04$ docker images hello
REPOSITORY   TAG        IMAGE ID       CREATED              SIZE
hello        version2   37e759ef1d58   About a minute ago   76.7kB
hello        version1   eca112c837bb   13 minutes ago       266MB
```

3- Expérience : retirez l'option -static du Dockerfile

```
 GNU nano 2.9.3                          Dockerfile.scratch

FROM alpine:3.20 AS build
RUN apk add --no-cache gcc musl-dev
COPY hello.c /src/hello.c
RUN gcc -o /hello /src/hello.c

FROM scratch
COPY --from=build /hello /hello
CMD ["/hello"]
```

```
chaima@chaima-VirtualBox:~/lab04$ docker build -f Dockerfile.scratch -t hello:test .
Sending build context to Docker daemon  4.096kB
Step 1/7 : FROM alpine:3.20 AS build
 ---> bf8527eb54c3
Step 2/7 : RUN apk add --no-cache gcc musl-dev
 ---> Using cache
 ---> e1283ad01d67
Step 3/7 : COPY hello.c /src/hello.c
 ---> Using cache
 ---> 81fdbbb94757
Step 4/7 : RUN gcc -o /hello /src/hello.c
 ---> Running in 201a84405100
Removing intermediate container 201a84405100
 ---> 2172ed1a1b14
Step 5/7 : FROM scratch
 --->
Step 6/7 : COPY --from=build /hello /hello
 ---> 7d8f3f92a65e
Step 7/7 : CMD ["/hello"]
 ---> Running in 6495e3e196fe
Removing intermediate container 6495e3e196fe
 ---> 8207b1134f6c
Successfully built 8207b1134f6c
Successfully tagged hello:test
```

```
chaima@chaima-VirtualBox:~/lab04$ docker run --rm hello:test
exec /hello: no such file or directory
```

Explication : le fichier /hello existe bien. Comme le binaire est maintenant dynamique, il a besoin au démarrage du chargeur /lib/ld-musl-x86_64.so.1 et de la libc musl, et l'image scratch est vide. Le noyau ne trouve pas ce chargeur, d'où le « no such file or directory ».

```
chaima@chaima-VirtualBox:~/lab04$ docker images hello
REPOSITORY   TAG        IMAGE ID       CREATED              SIZE
hello        test       8207b1134f6c   About a minute ago   18.4kB
hello        version2   37e759ef1d58   6 minutes ago        76.7kB
hello        version1   eca112c837bb   18 minutes ago       266MB
```

hello:test est encore plus petite que version2, parce que la libc n'est plus intégrée dans le binaire.

4- - Essayez d'ouvrir un shell dans l'image :

```
chaima@chaima-VirtualBox:~/lab04$ docker run --rm -it hello:version2 sh.
docker: Error response from daemon: failed to create shim task: OCI runtime create failed: runc
create failed: unable to start container process: exec: "sh.": executable file not found in $PAT
H: unknown.
```

```
chaima@chaima-VirtualBox:~/lab04$ docker create --name v2 hello:version2
677fd4044c6f757b3f233feac382f8d90e03a2ac4e04ae49f9018c3dec5d71fd
chaima@chaima-VirtualBox:~/lab04$ docker export v2 | tar -tv
-rwxr-xr-x 0/0               0 2026-11-07 13:24 .dockerenv
drwxr-xr-x 0/0               0 2026-11-07 13:24 dev/
-rwxr-xr-x 0/0               0 2026-11-07 13:24 dev/console
drwxr-xr-x 0/0               0 2026-11-07 13:24 dev/pts/
drwxr-xr-x 0/0               0 2026-11-07 13:24 dev/shm/
drwxr-xr-x 0/0               0 2026-11-07 13:24 etc/
-rwxr-xr-x 0/0               0 2026-11-07 13:24 etc/hostname
-rwxr-xr-x 0/0               0 2026-11-07 13:24 etc/hosts
lrwxrwxrwx 0/0               0 2026-11-07 13:24 etc/mtab -> /proc/mounts
-rwxr-xr-x 0/0               0 2026-11-07 13:24 etc/resolv.conf
-rwxr-xr-x 0/0           76664 2026-11-07 13:14 hello
drwxr-xr-x 0/0               0 2026-11-07 13:24 proc/
drwxr-xr-x 0/0               0 2026-11-07 13:24 sys/
chaima@chaima-VirtualBox:~/lab04$ docker rm v2
v2
```

Questions

1. Comparez la taille de hello:version1 et de hello:version2. D'où vient la différence ?
   → version1 fait environ 250 à 300 Mo (Ubuntu, gcc et libc-dev). version2 fait environ 100 Ko et ne contient que l'exécutable. Toute la chaîne de compilation est restée dans l'étape build, qui est jetée.
2. Expliquez l'erreur de l'expérience 3 : que cherche le programme au démarrage et qu'il ne trouve pas dans scratch ?
   → Un binaire dynamique a besoin, au lancement, de l'interpréteur ELF (ld-musl…) et de la bibliothèque C partagée. scratch ne contient rien de tout ça. -static intègre la libc dans le binaire.
3. Le compilateur gcc est-il présent dans l'image finale ? Comment le vérifier avec docker history ?
   → Non. docker history hello:version2 ne montre que la couche COPY /hello (environ 100 Ko) et le CMD. Aucune couche apk add gcc n'apparaît.
4. Citez un avantage de sécurité d'une image sans shell.
   → Un attaquant qui exploite le programme n'a ni shell, ni curl, ni gestionnaire de paquets pour continuer. La surface d'attaque et le nombre de CVE potentielles sont aussi minimes.

Partie 3 : image depuis un conteneur avec docker commit (version3)

1- Lancez un conteneur Ubuntu interactif nommé hello_container :

```
chaima@chaima-VirtualBox:~/lab04$ docker run -it --name hello_container ubuntu:24.04 bash
root@babd85f899fc:/#
```

2- Dans le conteneur (l'invite devient root@:/#), installez gcc et créez l'application :

```
root@babd85f899fc:/# apt-get update && apt-get install -y gcc
Get:1 http://archive.ubuntu.com/ubuntu noble InRelease [256 kB]
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [331 kB]
```

```
Setting up libc-devtools (2.39-0ubuntu8.9) ...
Setting up libheif-plugin-aomenc:amd64 (1.17.6-1ubuntu4.9) ...
Processing triggers for libc-bin (2.39-0ubuntu8.9) ...
root@babd85f899fc:/#
```

```
root@babd85f899fc:/# mkdir /hello_directory && cd /hello_directory
root@babd85f899fc:/hello_directory# cat > hello.c <<'EOF'
> #include <stdio.h>
> int main(void) {printf("Hello From Docker LAB !\n"); return0; }
> EOF
```

```
root@babd85f899fc:/hello_directory# gcc -o hello hello.c
root@babd85f899fc:/hello_directory# ./hello
Hello From Docker LAB !
root@babd85f899fc:/hello_directory# exit
exit
```

3- Sur la VM, listez les modifications faites dans le conteneur par rapport à son image (A = ajouté, C = modifié, D = supprimé) :

```
chaima@chaima-VirtualBox:~/lab04$ docker diff hello_container | grep hello
A /hello_directory
A /hello_directory/hello
A /hello_directory/hello.c
```

4- Créez l'image hello:version3 en définissant au passage la commande par défaut :

```
chaima@chaima-VirtualBox:~/lab04$ docker commit --change 'CMD ["/hello_directory/hello"]' hello_
container hello:version3
sha256:97c7e0625d2927765b6132baecbe7d8b148d7cb004ca02a2864f5f5eb24f47b1
chaima@chaima-VirtualBox:~/lab04$ docker run --rm hello:version3
Hello From Docker LAB !
```

5- Comparez l'historique des deux images Ubuntu :

```
chaima@chaima-VirtualBox:~/lab04$ docker history hello:version1
IMAGE          CREATED       CREATED BY                                      SIZE      COMMENT
eca112c837bb   2 hours ago   /bin/sh -c #(nop)  CMD ["/hello"]               0B
acc103a8e1f9   2 hours ago   /bin/sh -c gcc -o /hello /src/hello.c           16kB
c70a60aa5199   2 hours ago   /bin/sh -c #(nop) COPY file:616b7bfff4e63549…   95B
cbe421506eb0   2 hours ago   /bin/sh -c apt-get update && apt-get install…   188MB
d9b8c34d61a9   12 days ago   /bin/sh -c #(nop)  CMD ["/bin/bash"]            0B
<missing>      12 days ago   /bin/sh -c #(nop) ADD file:eed88e94d6aa7c9da…   78.2MB
<missing>      12 days ago   /bin/sh -c #(nop)  LABEL org.opencontainers.…   0B
<missing>      12 days ago   /bin/sh -c #(nop)  ARG LAUNCHPAD_BUILD_ARCH     0B
<missing>      12 days ago   /bin/sh -c #(nop)  ARG RELEASE                  0B
chaima@chaima-VirtualBox:~/lab04$ docker history hello:version3
IMAGE          CREATED              CREATED BY                                      SIZE      CO
MMENT
97c7e0625d29   About a minute ago   bash                                            261MB
d9b8c34d61a9   12 days ago          /bin/sh -c #(nop)  CMD ["/bin/bash"]            0B
<missing>      12 days ago          /bin/sh -c #(nop) ADD file:eed88e94d6aa7c9da…   78.2MB
<missing>      12 days ago          /bin/sh -c #(nop)  LABEL org.opencontainers.…   0B
<missing>      12 days ago          /bin/sh -c #(nop)  ARG LAUNCHPAD_BUILD_ARCH     0B
<missing>      12 days ago          /bin/sh -c #(nop)  ARG RELEASE                  0B
```

Comparaison des trois images :

```
chaima@chaima-VirtualBox:~/lab04$ docker images hello
REPOSITORY   TAG        IMAGE ID       CREATED              SIZE
hello        version3   97c7e0625d29   4 minutes ago        340MB
hello        version2   37e759ef1d58   About an hour ago    76.7kB
hello        version1   eca112c837bb   2 hours ago          266MB
```

Image	Méthode	Image de base	Taille relevée	Reproductible à l'identique ?
hello:version1	docker build	ubuntu:24.04	266 MB	**Oui** : le Dockerfile décrit chaque étape, on relance `docker build`
hello:version2	docker build multi-étapes	scratch	76,7 kB	**Oui** : Dockerfile, et l'image finale ne contient que le binaire
hello:version3	docker commit	ubuntu:24.04	*340 MB*	**Non** : les commandes ont été tapées à la main, rien ne les décrit

Questions :

1. Pourquoi hello:version3 est-elle plus lourde que hello:version1, alors que les deux partent d'Ubuntu ?
   → version3 est plus lourde car gcc a été installé **sans --no-install-recommends**, ce qui ajoute des paquets en plus, et **sans nettoyer le cache apt**. docker commit garde tout ce surplus.

2. Dans docker history hello:version3, peut-on savoir quelles commandes ont été tapées dans le conteneur ? Quelle conséquence pour la maintenance ?
   → Non. L'historique affiche une seule couche opaque (bash), sans les commandes tapées. L'image ne peut donc être ni auditée, ni corrigée, ni reconstruite facilement.
3. Un collègue doit reconstruire la même image dans six mois avec une version plus récente d'Ubuntu. Quelle méthode lui facilite la tâche, et pourquoi ?
   → Le **Dockerfile** : il suffit de changer la ligne FROM ubuntu:... puis de relancer docker build. Avec commit, il faudrait tout refaire à la main.

Partie 4 : publier l'image

4.1. Dans un registre local (sans compte)

1- Démarrez un registre privé sur le port 5000 :

```
chaima@chaima-VirtualBox:~/lab04$ docker run -d -p 5000:5000 --name registry registry:2
Unable to find image 'registry:2' locally
2: Pulling from library/registry
44cf07d57ee4: Pull complete
bbbdd6c6894b: Pull complete
8e82f80af0de: Pull complete
3493bf46cdec: Pull complete
6d464ea18732: Pull complete
Digest: sha256:a3d8aaa63ed8681a604f1dea0aa03f100d5895b6a58ace528858a7b332415373
Status: Downloaded newer image for registry:2
1c6195e92a0aac8994684aebd91a943e2b4767b03fdafc6d3bc9347c4a0612ec
```

2- Donnez à l'image un nom qui commence par l'adresse du registre, puis poussez-la :

```
chaima@chaima-VirtualBox:~/lab04$ docker tag hello:version2 localhost:5000/hello:2.0
chaima@chaima-VirtualBox:~/lab04$ docker push localhost:5000/hello:2.0
The push refers to repository [localhost:5000/hello]
f29bc481e8bd: Pushed
2.0: digest: sha256:1e82b1512bfd3071d543cd477eef24aabe225087f98cc02c0f168923c337fc28 size: 526
```

3- Consultez le contenu du registre :

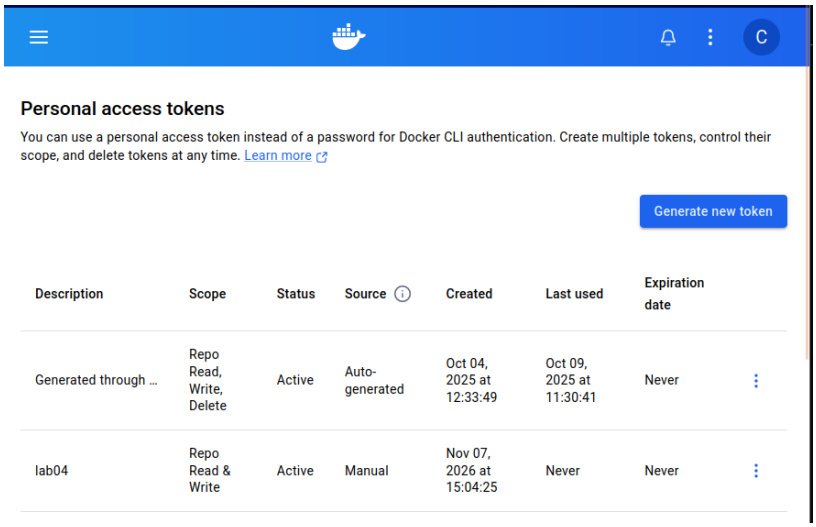
```
chaima@chaima-VirtualBox:~/lab04$ curl http://localhost:5000/v2/_catalog
{"repositories":["hello"]}
```

4- Supprimez les copies locales, puis relancez l'image :

```
chaima@chaima-VirtualBox:~/lab04$ docker rmi localhost:5000/hello:2.0 hello:version2
Untagged: localhost:5000/hello:2.0
Untagged: localhost:5000/hello@sha256:1e82b1512bfd3071d543cd477eef24aabe225087f98cc02c0f168923c3
37fc28
Untagged: hello:version2
Deleted: sha256:37e759ef1d583d2b5c11b8c0dcf5a4c913b08505c40d5a259d6decffcdb0bdc6
Deleted: sha256:2431f3bc86b8feda11aef5ab4be2f46294d533a8192bd0c4444a18c6b45ad668
Deleted: sha256:f29bc481e8bd282511c5979a45b0a21b4a4c624f1a9c3bc5e3c98457ba0bc6a4
chaima@chaima-VirtualBox:~/lab04$ docker run --rm localhost:5000/hello:2.0
Unable to find image 'localhost:5000/hello:2.0' locally
2.0: Pulling from hello
4bba1b4ec364: Pull complete
Digest: sha256:1e82b1512bfd3071d543cd477eef24aabe225087f98cc02c0f168923c337fc28
Status: Downloaded newer image for localhost:5000/hello:2.0
Hello From Docker LAB !
```

4.2. Sur Docker Hub (optionnel, compte nécessaire)

1- Créez un compte sur hub.docker.com :



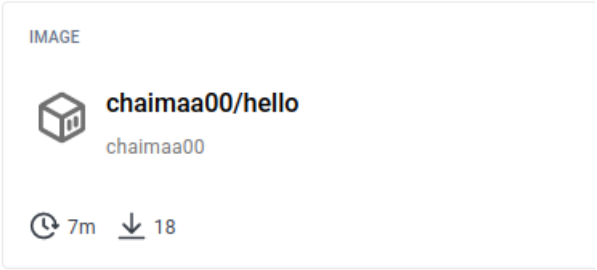
2- Connectez-vous, nommez l'image avec votre identifiant, puis poussez-la :

```
chaima@chaima-VirtualBox:~/lab04$ docker login -u chaimaa00
Password:
WARNING! Your password will be stored unencrypted in /home/chaima/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
```

```
chaima@chaima-VirtualBox:~/lab04$ docker tag localhost:5000/hello:2.0 chaimaa00/hello:2.0
chaima@chaima-VirtualBox:~/lab04$ docker push chaimaa00/hello:2.0
The push refers to repository [docker.io/chaimaa00/hello]
f29bc481e8bd: Pushed
2.0: digest: sha256:1e82b1512bfd3071d543cd477eef24aabe225087f98cc02c0f168923c337fc28 size: 526
```

3- Vérifiez sur le site que l'image apparaît dans vos dépôts.



Question : dans le nom localhost:5000/hello:2.0, identifiez le registre, le dépôt et le tag. Pourquoi docker push hello:2.0 ne suffirait-il pas ?

➔ **Registre** : localhost:5000
Dépôt : hello
Tag : 2.0
docker push hello:2.0 ne suffit pas, car le nom ne précise aucun registre. Docker envoie alors par défaut vers **Docker Hub**, dans le dépôt officiel docker.io/library/hello, où l'on n'a pas le droit d'écrire. C'est le nom de l'image qui indique **où** la pousser : localhost:5000/... pour le registre local, chaimaa00/... pour son compte Docker Hub. De plus, aucune image locale ne s'appelle hello:2.0.

Partie 5 : conteneuriser une application Python

1- Créez un nouveau répertoire et le fichier app.py :

```
chaima@chaima-VirtualBox:~/lab04$ mkdir -p ~/lab04/python && cd ~/lab04/python
chaima@chaima-VirtualBox:~/lab04/python$ cat > app.py <<'EOF'
> import platform
> print("Hello from Docker Python!")
> print("Version de Python :"),platform.python_version())
> EOF
```

2- Créez le Dockerfile :

```
chaima@chaima-VirtualBox:~/lab04/python$ cat > Dockerfile <<'EOF'
FROM python:3.12-slim
WORKDIR /app
COPY app.py .
CMD ["python", "app.py"]
EOF
```

3- Construisez et exécutez :

```
chaima@chaima-VirtualBox:~/lab04/python$ cat > app.py <<'EOF'
import platform
print("Hello from Docker Python!")
print("Version de Python :",platform.python_version())
EOF

chaima@chaima-VirtualBox:~/lab04/python$ cat app.py
import platform
print("Hello from Docker Python!")
print("Version de Python :",platform.python_version())
chaima@chaima-VirtualBox:~/lab04/python$ docker build -t python-hello:slim .
Sending build context to Docker daemon  3.072kB
Step 1/4 : FROM python:3.12-slim
 ---> 78976a69c067
Step 2/4 : WORKDIR /app
 ---> Using cache
 ---> be470fe31bb8
Step 3/4 : COPY app.py .
 ---> e362fe386c72
Step 4/4 : CMD ["python", "app.py"]
 ---> Running in 5352e40ec683
Removing intermediate container 5352e40ec683
 ---> 2a5ed88f9096
Successfully built 2a5ed88f9096
Successfully tagged python-hello:slim
chaima@chaima-VirtualBox:~/lab04/python$ docker run --rm python-hello:slim
Hello from Docker Python!
Version de Python : 3.12.16
```

4- Modifiez la première ligne du Dockerfile en FROM python:3.12, reconstruisez avec le tag python hello:full, puis comparez :

```
chaima@chaima-VirtualBox:~/lab04/python$ docker build -t python-hello:full .
Sending build context to Docker daemon  3.072kB
Step 1/4 : FROM python:3.12
3.12: Pulling from library/python
a6dd1c05fbba: Pull complete
e8a88afca2bd: Pull complete
411d2dc8717d: Pull complete
696c1fe91c64: Pull complete
5ac0faf5954b: Pull complete
4ed680595f4b: Pull complete
685e2037ccfc: Pull complete
Digest: sha256:cad37dd20c7cac861af9003af8796d03ea7a998ec1e3a664429740fc9d7d269
Status: Downloaded newer image for python:3.12
 ---> 37662cc6e541
Step 2/4 : WORKDIR /app
 ---> Running in ce9ed5883f28
Removing intermediate container ce9ed5883f28
 ---> a63ea620ec61
Step 3/4 : COPY app.py .
 ---> de73579215da
Step 4/4 : CMD ["python", "app.py"]
 ---> Running in f2df312ce270
Removing intermediate container f2df312ce270
 ---> 73566a2377a0
Successfully built 73566a2377a0
Successfully tagged python-hello:full
```

5- Remplacez la commande par défaut au lancement :

```
chaima@chaima-VirtualBox:~/lab04/python$ docker run --rm python-hello:slim python -c
 "print(2 + 2)"
4
```

Questions :

1. Quel est l'écart de taille entre slim et full ? Que contient en plus l'image complète ?
   → slim fait environ 120–150 MB et full environ 1 GB (reporte tes valeurs). L'image complète contient en plus des outils de compilation (gcc, make), des en-têtes de développement, git, curl et de nombreuses bibliothèques, utiles pour installer des modules Python à compiler.
2. À l'étape 5, qu'est devenu le CMD du Dockerfile ?
   → Le CMD a été **remplacé** par la commande donnée après le nom de l'image. app.py n'est donc pas exécuté, seul print(2 + 2) l'est.
3. Pourquoi la version 2025 du lab utilisait-elle python:3.9, et pourquoi faut-il la changer aujourd'hui ?
   → Python 3.9 était une version encore maintenue en 2025. Elle est en **fin de vie depuis octobre 2025** et ne reçoit plus de correctifs de sécurité, d'où le passage à une version maintenue comme 3.12.

Questions de compte rendu

1. Classez les images hello:version1, version2 et version3 de la plus petite à la plus grande, et expliquez chaque écart.
   → **Classement : version2 (76,7 kB) < version1 (266 MB) < version3 340 MB)**

- **version2** est la plus petite : basée sur scratch (vide), elle ne contient que le binaire statique. gcc est resté dans l'étape de build, qui a été jetée.
- **version1** contient Ubuntu, gcc et libc6-dev installés avec --no-install-recommends, et le cache apt a été supprimé.
- **version3** est la plus lourde : elle contient Ubuntu et gcc **avec** tous les paquets recommandés, et le cache apt n'a pas été nettoyé. docker commit a tout gardé.

2. Donnez deux raisons de préférer un Dockerfile à docker commit dans une équipe.
   - **Traçabilité** : le Dockerfile est un fichier texte versionné (Git) que toute l'équipe peut lire, relire et auditer.
   - **Reproductibilité** : n'importe qui reconstruit la même image avec docker build, et la mettre à jour (nouvelle base, nouveau paquet) ne demande que de modifier une ligne.
3. Dans quel cas docker commit reste-t-il utile ? (Indice : dépannage, exploration.)
   → Pour le **dépannage** (figer l'état d'un conteneur en panne afin de l'analyser plus tard) et l'**exploration** (sauvegarder un essai manuel avant d'écrire le vrai Dockerfile).
4. Expliquez le rôle du point . à la fin de docker build. Que se passerait-il si on lançait la commande depuis le répertoire personnel ?
   → C'est le **contexte de build** : le dossier envoyé au moteur Docker, le seul où COPY peut lire des fichiers. Depuis ~, tout le répertoire personnel serait envoyé, ce qui rend le build lent et risque d'inclure des fichiers sensibles. De plus, Dockerfile.ubuntu ne serait pas trouvé, et COPY hello.c échouerait, car le fichier est dans lab04/ et non à la racine du contexte.
5. Qu'apporte le build multi-étapes par rapport à la compilation sur l'hôte proposée dans la version 2025 du lab ?
   → Plus besoin d'installer gcc ni les bibliothèques statiques sur la VM. Toute la compilation est décrite dans le Dockerfile, donc le build est **identique sur n'importe quelle machine** qui a Docker. L'image finale reste minimale, car seul le binaire est copié.
6. Nettoyez votre environnement et notez l'espace libéré :

```
chaima@chaima-VirtualBox:~/lab04/python$ docker rm -f hello_container registry
hello_container
registry
chaima@chaima-VirtualBox:~/lab04/python$ docker system df
TYPE            TOTAL     ACTIVE    SIZE      RECLAIMABLE
Images          21        7         2.436GB   2.215GB (90%)
Containers      10        0         148.1kB   148.1kB (100%)
Local Volumes   1         0         29.42kB   29.42kB (100%)
Build Cache     0         0         0B        0B
chaima@chaima-VirtualBox:~/lab04/python$ docker image prune
WARNING! This will remove all dangling images.
Are you sure you want to continue? [y/N] y
Deleted Images:
deleted: sha256:d1799e3c7b8cb815e62366547d475eac823dd9097bd0d15a73f70dd3e4bb862b
deleted: sha256:3e56c433cdadcc2bbc3ff2db46cab606256c4780b80f29b6f85f455bb61f9bb1
deleted: sha256:2172ed1a1b14e4420a5ed7a5f7531028b6540fd115b1b099484fb90084edc962
deleted: sha256:ad314517f47745994e2b34a2a677995d71e46a11738c9652c6670e4c1e19814b
deleted: sha256:81fdbbb947572cc310113adbe5e627fe91d874c9491d22759cdf54ad137d04d8
deleted: sha256:a42dd2aed5770af09450fef5cde474ec1e84ee571762b4ef28bf9158df101631
deleted: sha256:e1283ad01d6705d7ec529cf49b885ce2c440025d11deacd70d35e0d65966127a
deleted: sha256:2da8229d10de30b1fa3b39b46b04e7d9fb10ed8e2099c61ade977ac60e476108
deleted: sha256:29963cead35fc1aa37c0e033afa593f0542c193f7098df9d2d79348227f66480
deleted: sha256:0a5936b8dae031389e573efcf240e0f626a52a42d4e484770f3f6393fb12c1e9
deleted: sha256:f53abf798290f5d9e462b1260b7fdda3a3eb22beaf4a67d0c1ff3b9089748b03

Total reclaimed space: 155MB
chaima@chaima-VirtualBox:~/lab04/python$
```